Alternative Assessment Tool (AAT) Report on

"Design and Verification of Combinational and Sequential Digital Modules using Verilog HDL"

Submitted in partial fulfilment of the requirements of the Alternative Assessment Tool (AAT) for the course

ADVANCED DIGITAL AND LOGIC DESIGN (ADLD)

[Course Code: 23ECSPE1AD]

Submitted by

NAVEEN B **1BM25EC429**

Submitted to

Prof. Ajaykumar Devarapalli

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

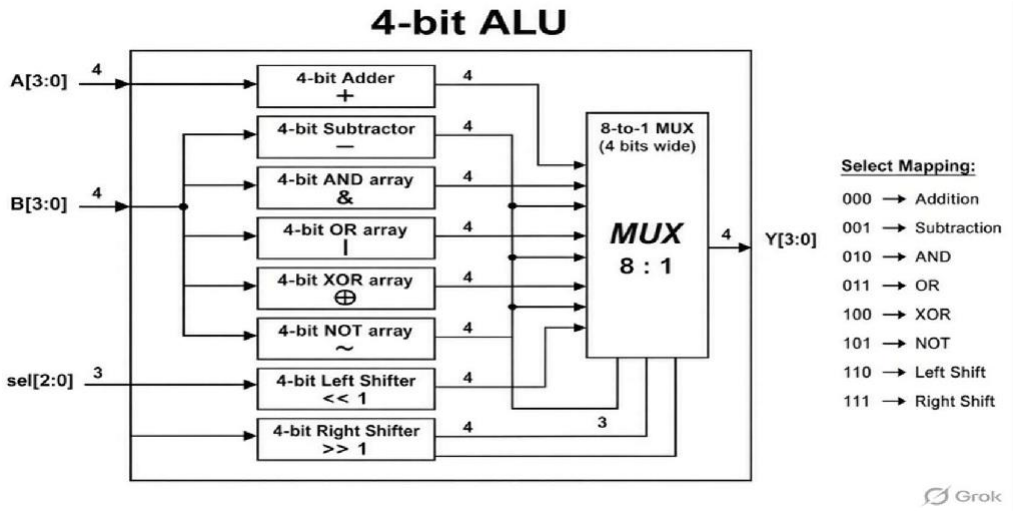
BMS College of Engineering
Bull Temple Road, Basavanagudi, Bengaluru-560019

Academic Year

2026 – 2027

1 DESIGN, VERILOG HDL CODE & TESTBENCH FOR ARITHEMATIC LOGIC UNIT:

DESIGN



VERILOG HDL CODE & TESTBENCH

```
`timescale 1ns/1ps

module alu_tb;

reg  [3:0] A;
reg  [3:0] B;
reg  [2:0] sel;
wire [3:0] Y;

alu uut (
    .A(A),
    .B(B),
    .sel(sel),
    .Y(Y)
);

initial begin
    $dumpfile("alu.vcd");
    $dumpvars(0, alu_tb);

    $display("A    B    sel   Y");
    $monitor("%b  %b  %b  %b", A, B, sel, Y);

    A = 4'b0101;
    B = 4'b0011;

    sel = 3'b000; #10;  // Addition
    sel = 3'b001; #10;  // Subtraction
    sel = 3'b010; #10;  // AND
    sel = 3'b011; #10;  // OR
    sel = 3'b100; #10;  // XOR
    sel = 3'b101; #10;  // NOT
    sel = 3'b110; #10;  // Left shift
    sel = 3'b111; #10;  // Right shift

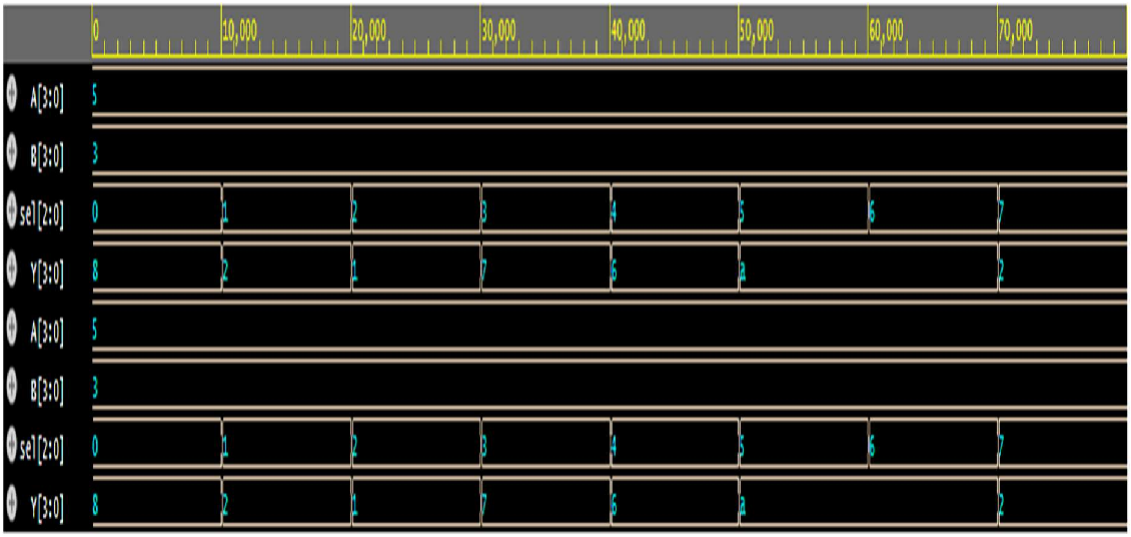
    $finish;
end
```

```
module alu (
    input  [3:0] A,
    input  [3:0] B,
    input  [2:0] sel,
    output reg [3:0] Y
);

always @(*) begin
    case (sel)
        3'b000: Y = A + B;   // Addition
        3'b001: Y = A - B;   // Subtraction
        3'b010: Y = A & B;   // AND
        3'b011: Y = A | B;   // OR
        3'b100: Y = A ^ B;   // XOR
        3'b101: Y = ~A;      // NOT
        3'b110: Y = A << 1;  // Left shift
        3'b111: Y = A >> 1;  // Right shift
        default: Y = 4'b0000;
    endcase
end

endmodule
```

OUTPUT



- An ALU (Arithmetic Logic Unit) is a digital circuit that performs arithmetic and logical operations on binary data. It is a basic component of a CPU or processor.

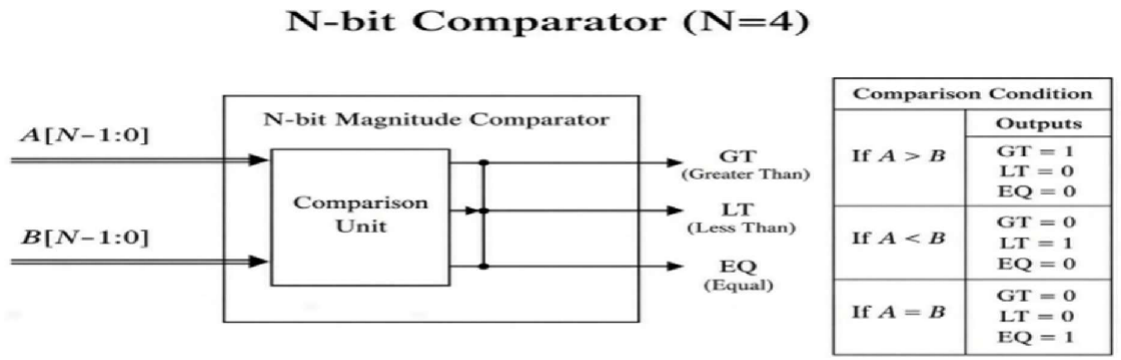
- The ALU takes two 4-bit inputs, A and B, and a 3-bit selection input, sel. Based on sel, it performs one of eight operations.

Output Waveform Explanation:

- input [3:0] A, B: Two 4-bit input operands.
- input [2:0] sel: Selects one of eight operations.
- output reg [3:0] Y: Stores the result of the selected operation.
- always @(*): Executes the combinational logic whenever an input changes.
- case (sel): Selects the operation according to the selection code.
- default: Provides a default output and helps avoid unintended latch inference.

2 DESIGN, VERILOG HDL CODE & TESTBENCH FOR N-BIT COMPARATOR:

DESIGN



VERILOG HDL CODE & TESTBENCH

```
module tb;
parameter N = 4;
reg  [N-1:0] A, B;
wire GT, LT, EQ;
n_bit_comparator #(N) dut (
    .A(A),
    .B(B),
    .GT(GT),
    .LT(LT),
    .EQ(EQ)
);
initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, tb);

    $monitor("Time=%0t A=%b B=%b GT=%b LT=%b EQ=%b",
             $time, A, B, GT, LT, EQ);

    A = 4'b1010; B = 4'b0101; #10;
    A = 4'b0011; B = 4'b1001; #10;
    A = 4'b0110; B = 4'b0110; #10;
    A = 4'b1111; B = 4'b0001; #10;
    A = 4'b0000; B = 4'b1111; #10;

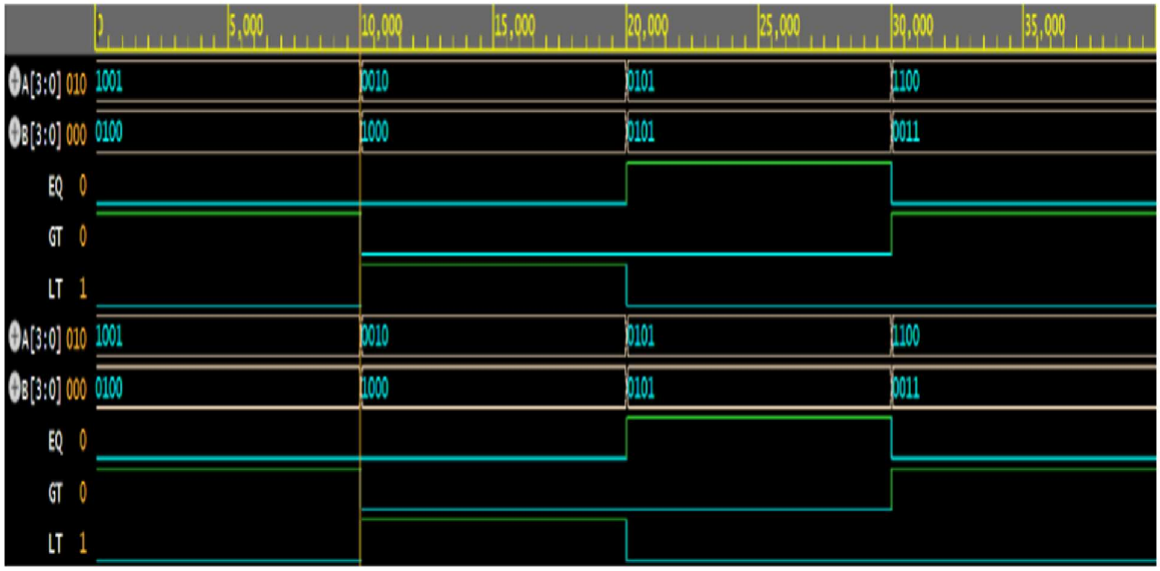
    $finish;
end
endmodule
```

```
module n_bit_comparator #(parameter N = 4) (
    input  [N-1:0] A,
    input  [N-1:0] B,
    output reg GT,
    output reg LT,
    output reg EQ
);

always @(*) begin
    if (A > B) begin
        GT = 1'b1;
        LT = 1'b0;
        EQ = 1'b0;
    end
    else if (A < B) begin
        GT = 1'b0;
        LT = 1'b1;
        EQ = 1'b0;
    end
    else begin
        GT = 1'b0;
        LT = 1'b0;
        EQ = 1'b1;
    end
end

endmodule
```

OUTPUT



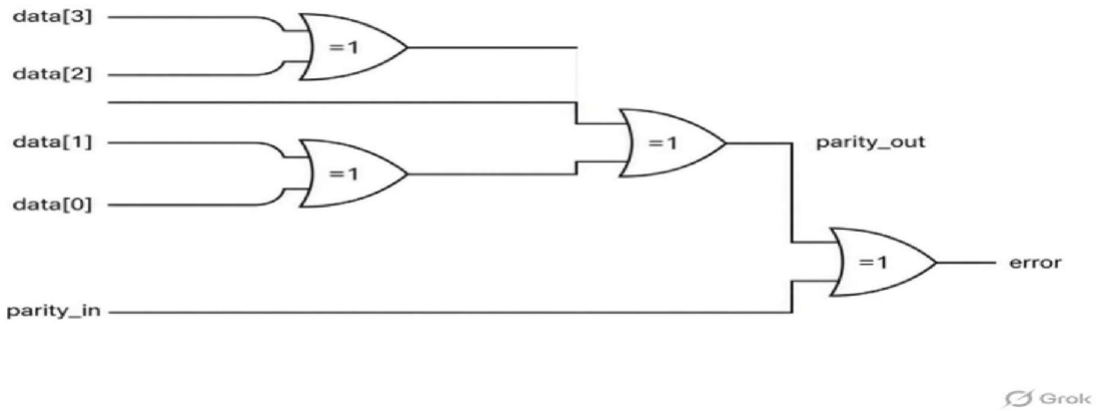
- A comparator is a combinational logic circuit used to compare two binary numbers. The N-bit comparator has two N-bit inputs, A and B, and three outputs: GT, LT, and EQ. The parameter N allows the width of the inputs to be changed.

Output Waveform Explanation:

The waveform displays the input signals A and B and the output signals GT, LT, and EQ. When A is greater than B, GT becomes high. When A is less than B, LT becomes high. When both inputs are equal, EQ becomes high. For each comparison, only one output should be high.

3 DESIGN, VERILOG HDL CODE & TESTBENCH FOR 4-BIT PARITY GENERATOR AND CHECKER:

DESIGN



VERILOG HDL CODE & TESTBENCH

```
module tb;
    reg [3:0] data;
    reg parity_in;
    wire parity_out;
    wire error;
    parity uut (
        .data(data),
        .parity_in(parity_in),
        .parity_out(parity_out),
        .error(error)
    );
    initial begin
        $dumpfile("parity.vcd");
        $dumpvars(0, tb);
        data = 4'b0000; parity_in = 0;
        #10;data = 4'b0001; parity_in = 1;
        #10;
        data = 4'b0011; parity_in = 0;
        #10;data = 4'b1010; parity_in = 0;
        #10;
        data = 4'b1010; parity_in = 1;
        #10;
        $finish;
    end
    initial begin
        $monitor("Time=%0t data=%b parity_in=%b
parity_out=%b error=%b",
                 $time, data, parity_in, parity_out,
error);
    end
endmodule
```

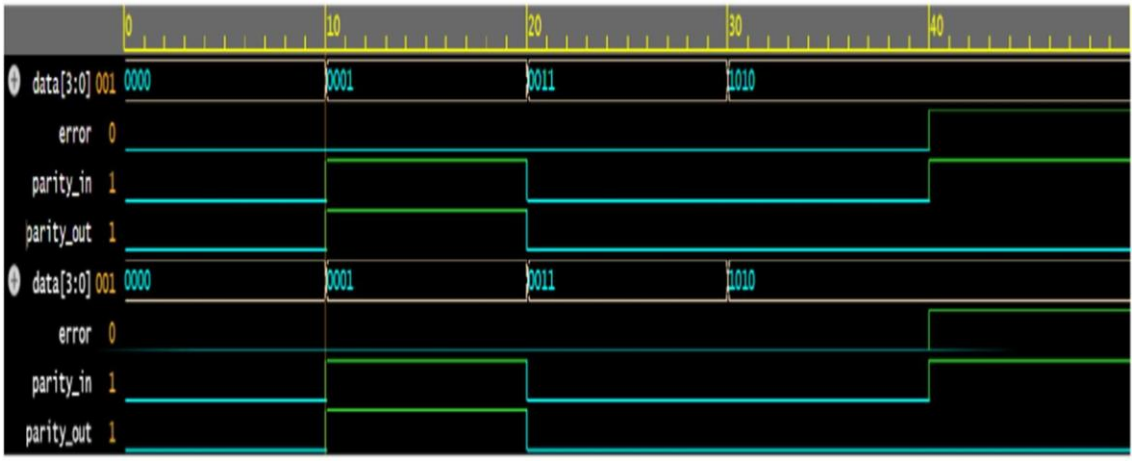
```
module parity (
    input [3:0] data,
    input parity_in,
    output reg parity_out,
    output reg error
);

    always @(*) begin
        // Even parity generator
        parity_out = ^data;

        // Parity checker
        if ((^data) == parity_in)
            error = 1'b0;
        else
            error = 1'b1;
    end

endmodule
```

OUTPUT



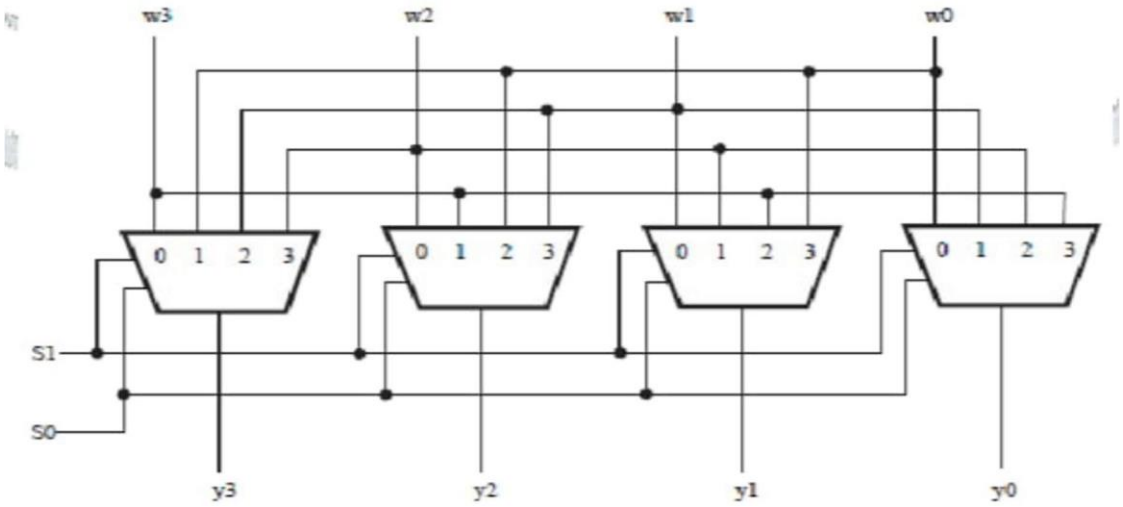
- The parity generator calculates the parity bit using the XOR operation on the 4-bit input data. The parity checker compares the calculated parity with the received parity_in bit. If both values match, error = 0; otherwise, error = 1.

Output Waveform Explanation:

The waveform shows the input signals data and parity_in, along with the output signals parity_out and error. For data = 0000 and parity_in = 0, the error output is 0. For data = 0001 and parity_in = 1, the error output is also 0. For data = 0011 and parity_in = 0, the error output is 0. For data = 1010 and parity_in = 0, the error output is 0. Finally, for data = 1010 and parity_in = 1, the error output is 1.

4 DESIGN, VERILOG HDL CODE & TESTBENCH FOR 4-BIT BARREL SHIFTER:

DESIGN



VERILOG HDL CODE & TESTBENCH

```
module tb;

reg [3:0] w;
reg [1:0] sel;
wire [3:0] y;

barrel_shifter dut (
    .w(w),
    .sel(sel),
    .y(y)
);

initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, tb);

    w = 4'b1010; sel = 2'b00; #10;
    w = 4'b1010; sel = 2'b01; #10;
    w = 4'b1010; sel = 2'b10; #10;
    w = 4'b1010; sel = 2'b11; #10;

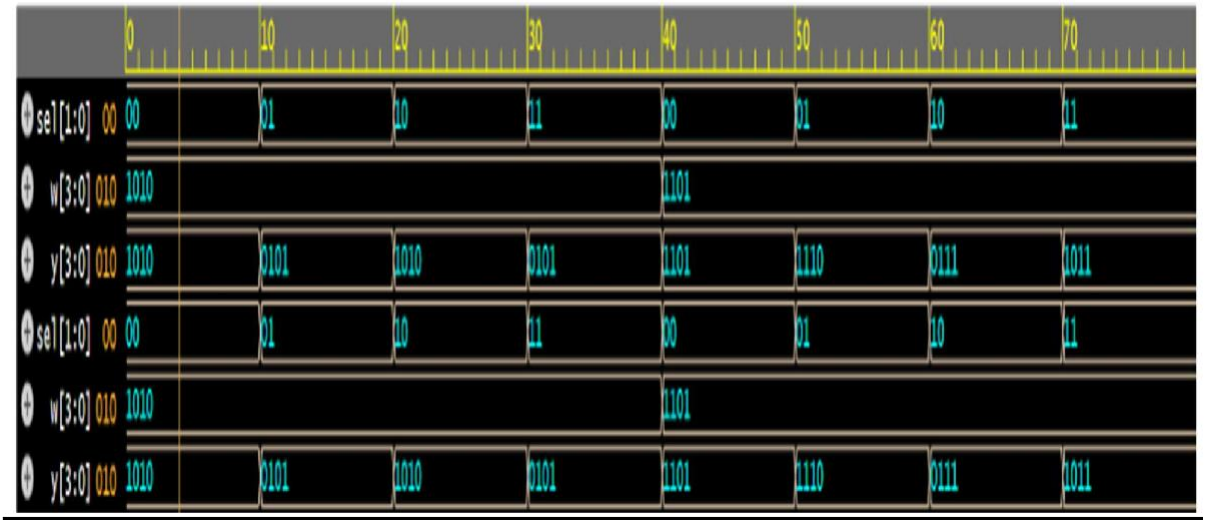
    w = 4'b1101; sel = 2'b00; #10;
    w = 4'b1101; sel = 2'b01; #10;
    w = 4'b1101; sel = 2'b10; #10;
    w = 4'b1101; sel = 2'b11; #10;
```

```
module barrel_shifter (
    input [3:0] w,
    input [1:0] sel,
    output reg [3:0] y
);

always @(*) begin
    case (sel)
        2'b00: y = w;
        2'b01: y = {w[0], w[3], w[2], w[1]};
        2'b10: y = {w[1], w[0], w[3], w[2]};
        2'b11: y = {w[2], w[1], w[0], w[3]};
    endcase
end

endmodule
```

OUTPUT



- A barrel shifter is a combinational circuit that shifts or rotates binary data by a specified number of bit positions. In this experiment, a 4-bit circular right-rotation operation is performed based on the 2-bit select input sel.

Output Waveform Explanation:

The waveform shows the input data w, select input sel, and output y. When sel = 00, the output remains unchanged. When sel = 01, the data rotates right by one bit. When sel = 10, it rotates right by two bits. When sel = 11, it rotates right by three bits. The output changes according to the select input.

5 DESIGN, VERILOG HDL CODE & TESTBENCH FOR 4-BIT BARREL SHIFTER:

DESIGN



VERILOG HDL CODE & TESTBENCH

```
`timescale 1ns/1ps

module timer_tb;

reg clk;
reg rst;
reg start;
reg [3:0] load_value;

wire [3:0] count;
wire done;

timer uut (
    .clk(clk),
    .rst(rst),
    .start(start),
    .load_value(load_value),
    .count(count),
    .done(done)
);

// Generate clock with 10 ns period
initial begin
    clk = 0;
    forever #5 clk = ~clk;
end

initial begin
    $dumpfile("timer.vcd");
    $dumpvars(0, timer_tb);

    $monitor("Time=%0t rst=%b start=%b count=%d done=%b",
             $time, rst, start, count, done);

    // Apply reset
    rst = 1;
    start = 0;
    load_value = 4'd5;

    @(negedge clk);
    rst = 0;

    // Start countdown from 5
    start = 1;

    @(negedge clk);
    start = 0;

    // Allow countdown to complete
    repeat (7) @(negedge clk);

    $finish;
end

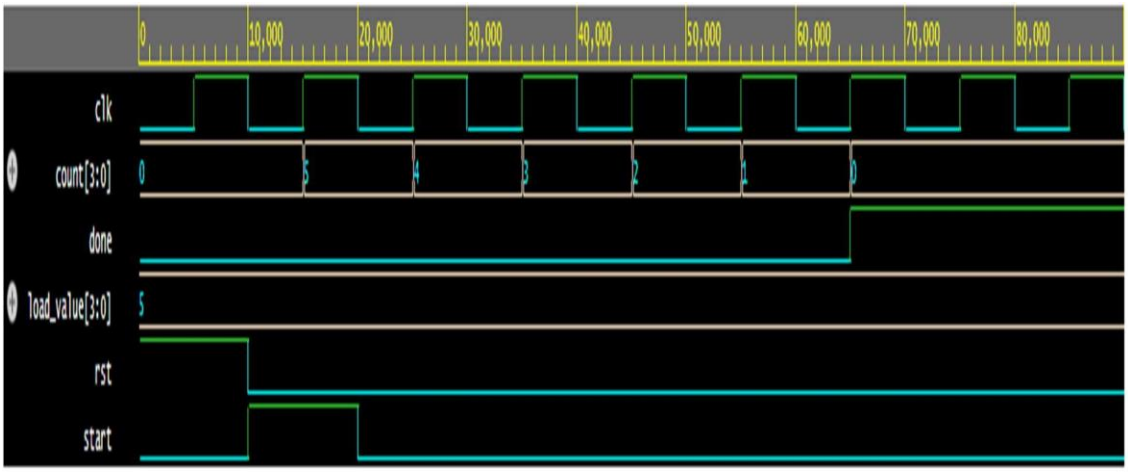
endmodule
```

```
module timer (
    input clk,
    input rst,
    input start,
    input [3:0] load_value,
    output reg [3:0] count,
    output reg done
);

always @(posedge clk) begin
    if (rst) begin
        count <= 4'b0000;
        done  <= 1'b0;
    end
    else if (start) begin
        count <= load_value;
        done  <= (load_value == 0);
    end
    else if (count > 0) begin
        if (count == 1) begin
            count <= 4'b0000;
            done  <= 1'b1;
        end
        else begin
            count <= count - 1'b1;
        end
    end
end

endmodule
```

OUTPUT



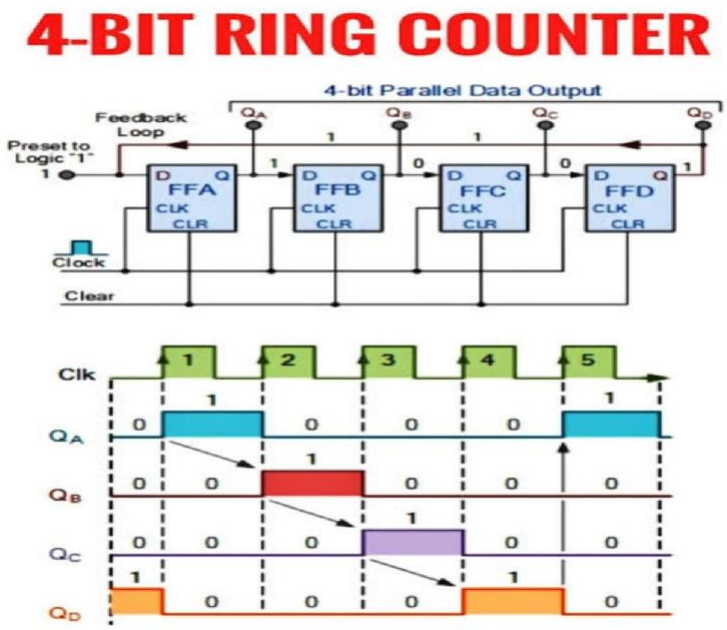
- Here is a complete 4-bit countdown timer module, including the design, Verilog HDL code, and testbench. It is suitable for running on EDA Playground.
- A timer module is a digital circuit that counts clock pulses to measure time or generate a delay. In this example, we will design a countdown timer that starts from a loaded value and counts down to zero.

Output Waveform Explanation:

- always @(posedge clk) — updates the timer at each rising clock edge.
- rst — clears the count and the done signal.
- start — loads the starting value into the counter.
- count - 1'b1 — decrements the counter by one.
- count == 1 — sets the count to zero and asserts done.
- When the count is zero, it stays there until another start or reset operation.

6 DESIGN, VERILOG HDL CODE & TESTBENCH FOR RING COUNTER:

DESIGN

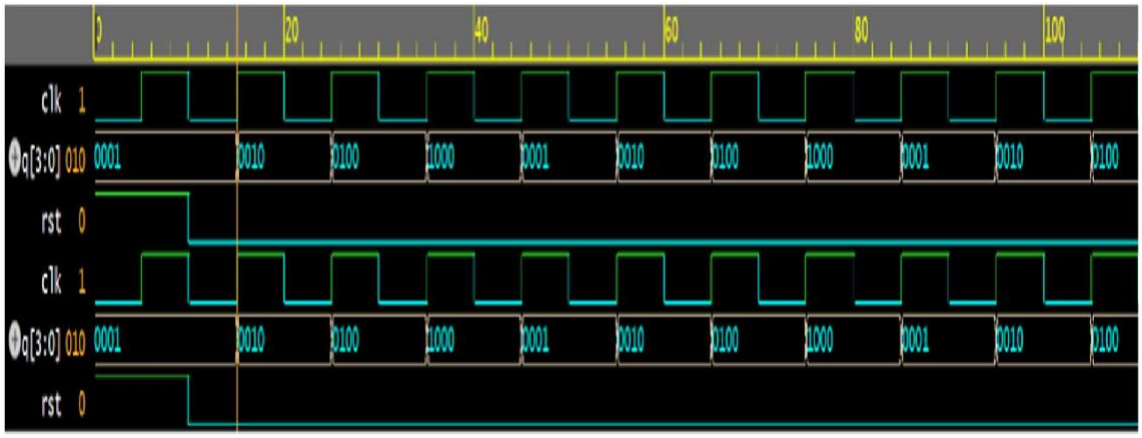


VERILOG HDL CODE & TESTBENCH

```
module tb;
    reg clk, rst;
    wire [3:0] q;
    ring_counter uut (
        .clk(clk),
        .rst(rst),
        .q(q)
    );
    initial begin
        $dumpfile("ring_counter.vcd");
        $dumpvars(0, tb);
        clk = 0;
        rst = 1;
        #10;
        rst = 0;
        #100;
        $finish;
    end
    always #5 clk = ~clk;
    initial begin
        $monitor("Time=%0t clk=%b rst=%b q=%b",
                 $time, clk, rst, q);
    end
endmodule
```

```
module ring_counter (
    input clk,
    input rst,
    output reg [3:0] q
);
    always @(posedge clk or posedge rst) begin
        if (rst)
            q <= 4'b0001;
        else
            q <= {q[2:0], q[3]};
    end
endmodule
```

OUTPUT



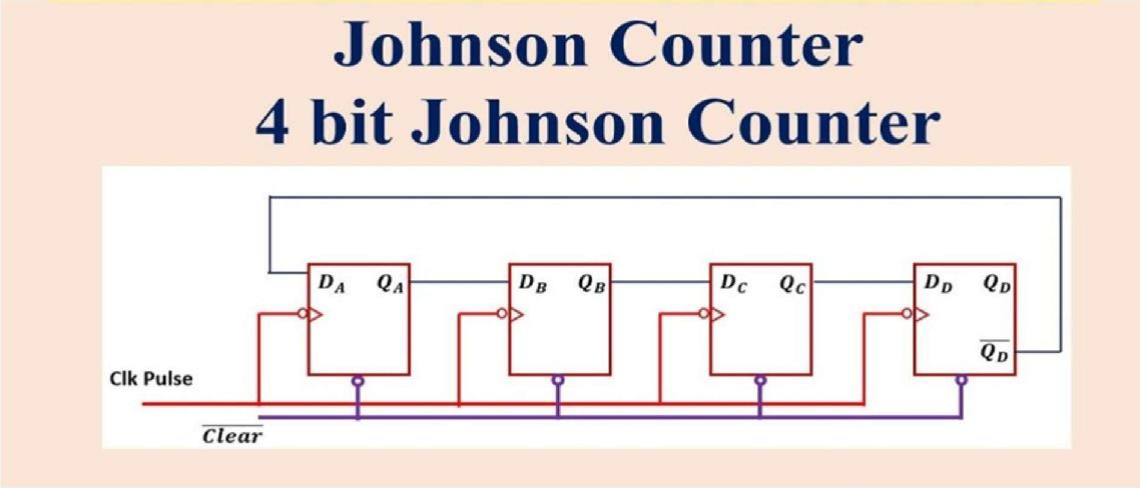
A 4-bit ring counter is a sequential circuit in which the output of the last flip-flop is fed back to the first flip-flop. The counter uses a clock signal to shift a single logic 1 through the four bits. The reset initializes the output to 0001.

Output Waveform Explanation:

The waveform shows the clock, reset, and 4-bit output. After reset is released, the output shifts at every rising edge of the clock in the sequence 0001 → 0010 → 0100 → 1000 → 0001. This sequence repeats continuously, verifying the correct operation of the ring counter.

7 DESIGN, VERILOG HDL CODE & TESTBENCH FOR JOHNSON COUNTER:

DESIGN

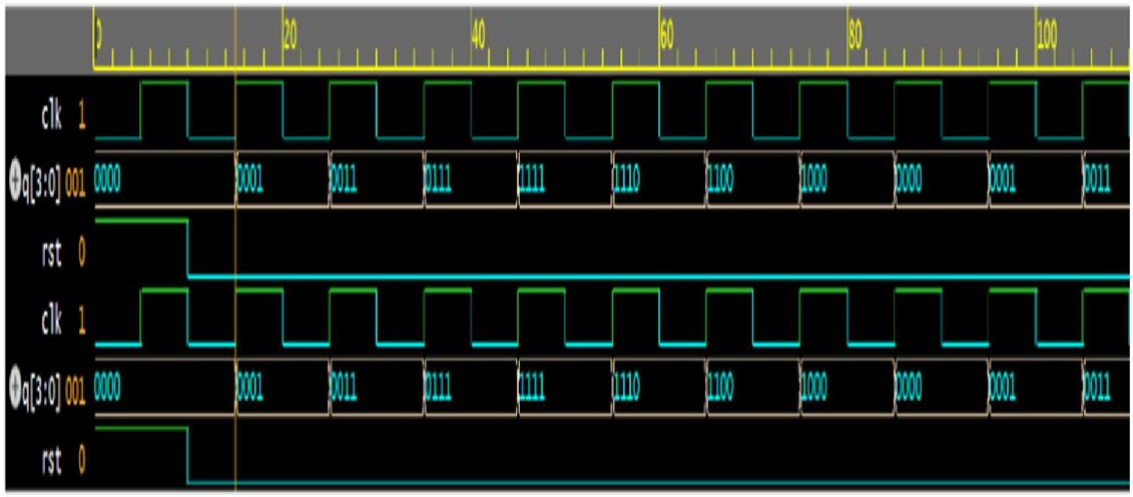


VERILOG HDL CODE & TESTBENCH

```
module tb;
    reg clk, rst;
    wire [3:0] q;
    johnson_counter uut (
        .clk(clk),
        .rst(rst),
        .q(q)
    );
    initial begin
        $dumpfile("johnson_counter.vcd");
        $dumpvars(0, tb);
        clk = 0;
        rst = 1;
        #10;
        rst = 0;
        #100;
        $finish;
    end
    always #5 clk = ~clk;
    initial begin
        $monitor("Time=%0t clk=%b rst=%b q=%b",
                 $time, clk, rst, q);
    end
endmodule
```

```
module johnson_counter (
    input clk,
    input rst,
    output reg [3:0] q
);
    always @(posedge clk or posedge rst) begin
        if (rst)
            q <= 4'b0000;
        else
            q <= {q[2:0], ~q[3]};
    end
endmodule
```

OUTPUT



- A 4-bit Johnson counter is a sequential circuit in which the inverted output of the last flip-flop is fed back to the first flip-flop. It uses a clock signal to shift the bits and produces eight unique states.

Output Waveform Explanation:

The waveform shows the clock, reset, and 4-bit output. After reset is released, the output changes at each rising edge of the clock in the sequence:

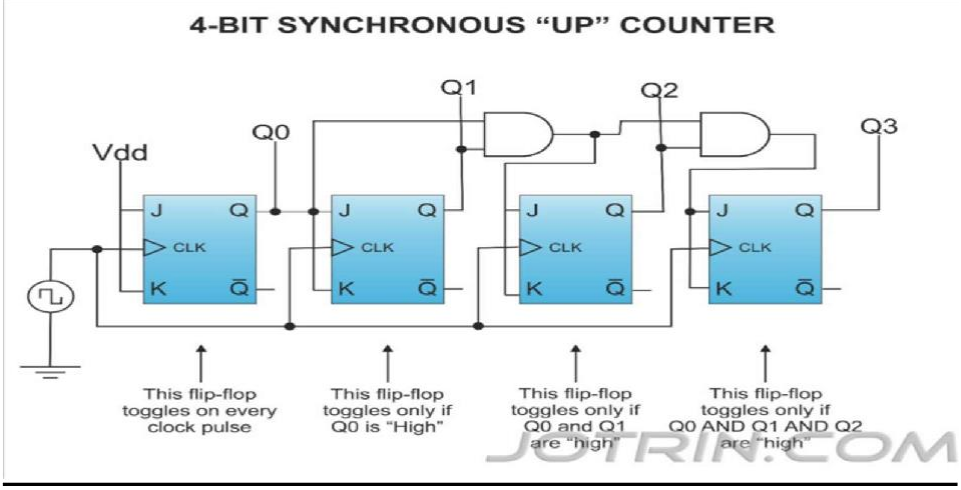
0000 → 0001 → 0011 → 0111 → 1111 → 1110 → 1100 → 1000 → 0000

The sequence repeats continuously, verifying the correct operation of the 4-bit Johnson counter.

8 DESIGN, VERILOG HDL CODE & TESTBENCH FOR SYNCHRONOUS COUNTER AND ASYNCHRONOUS COUNTER :

SYNCHRONOUS COUNTER

DESIGN

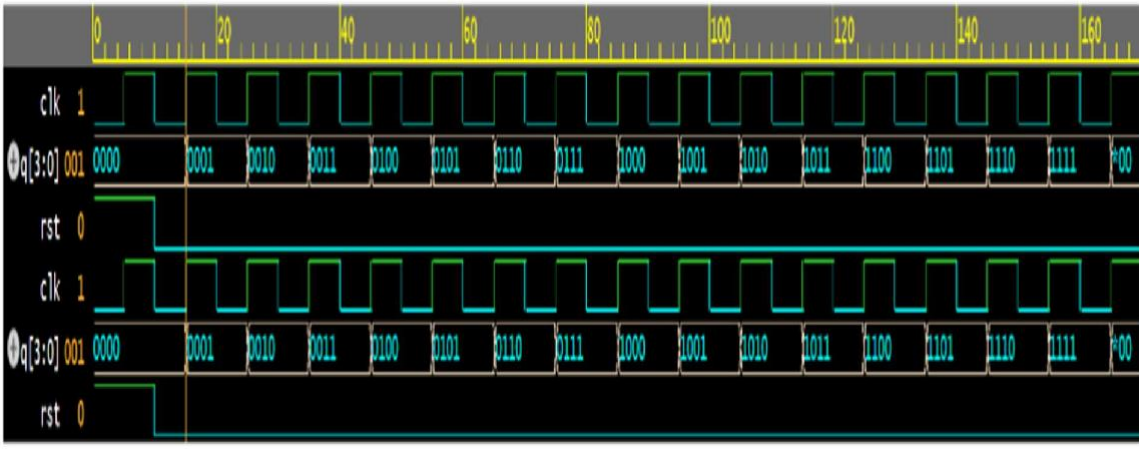


VERILOG HDL CODE & TESTBENCH

```
module tb;
    reg clk, rst;
    wire [3:0] q;
    sync_counter uut (
        .clk(clk),
        .rst(rst),
        .q(q)
    );
    initial begin
        $dumpfile("sync_counter.vcd");
        $dumpvars(0, tb);
        clk = 0;
        rst = 1;
        #10;
        rst = 0;
        #160;
        $finish;
    end
    always #5 clk = ~clk;
    initial begin
        $monitor("Time=%0t clk=%b rst=%b q=%b",
                 $time, clk, rst, q);
    end
endmodule
```

```
module sync_counter (
    input clk,
    input rst,
    output reg [3:0] q
);
    always @(posedge clk or posedge rst) begin
        if (rst)
            q <= 4'b0000;
        else
            q <= q + 1'b1;
    end
endmodule
```

OUTPUT



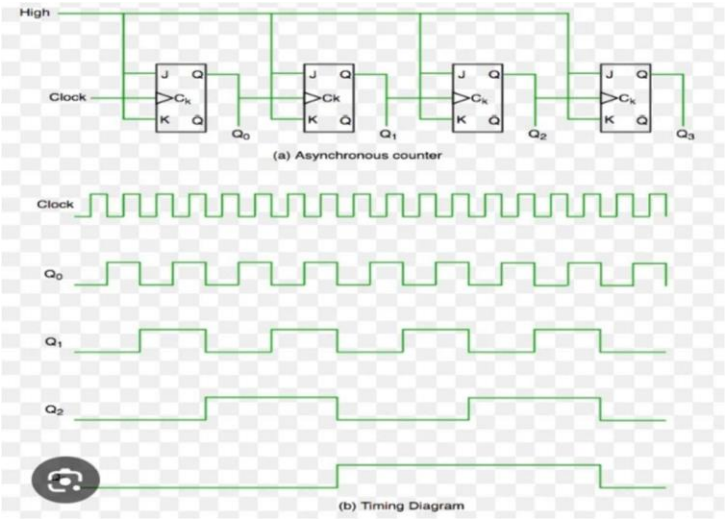
- A synchronous counter is a sequential circuit in which all flip-flops receive the same clock signal. The counter increments its binary value at every rising edge of the clock. The reset initializes the output to 0000

Output Waveform Explanation:

The waveform shows the clock, reset, and 4-bit output. After reset is released, the output increments from 0000 to 0001, 0010, 0011, and continues in binary order at every rising clock edge.

ASYNCHRONOUS COUNTER

DESIGN

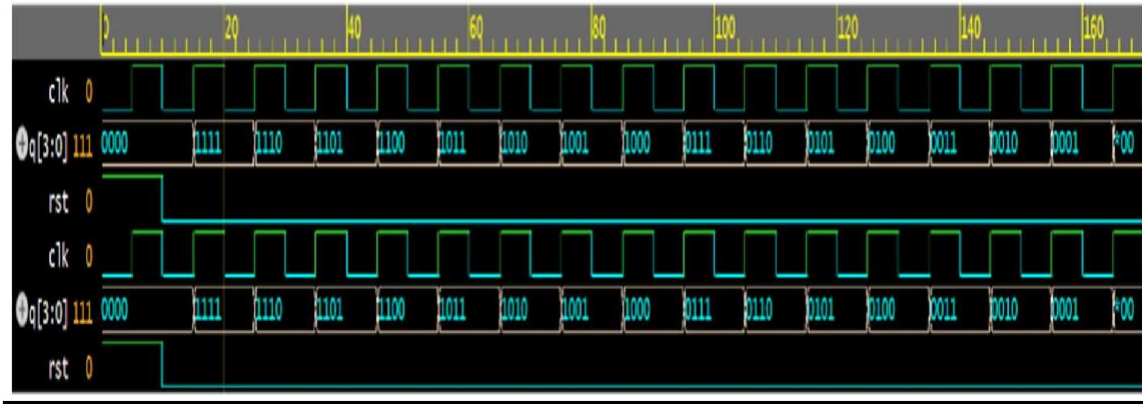


VERILOG HDL CODE & TESTBENCH

```
module tb;
    reg clk, rst;
    wire [3:0] q;
    async_counter uut (
        .clk(clk),
        .rst(rst),
        .q(q)
    );
    initial begin
        $dumpfile("async_counter.vcd");
        $dumpvars(0, tb);
        clk = 0;
        rst = 1;
        #10;
        rst = 0;
        #160;
        $finish;
    end
    always #5 clk = ~clk;
    initial begin
        $monitor("Time=%0t clk=%b rst=%b q=%b",
                 $time, clk, rst, q);
    end
endmodule
```

```
module async_counter (
    input clk,
    input rst,
    output reg [3:0] q
);
  always @(posedge clk or posedge rst) begin
        if (rst)
            q[0] <= 1'b0;
        else
            q[0] <= ~q[0];end
    always @(posedge q[0] or posedge rst) begin
        if (rst)
            q[1] <= 1'b0;
        else
            q[1] <= ~q[1]; end
    always @(posedge q[1] or posedge rst) begin
        if (rst)
            q[2] <= 1'b0;
        else
            q[2] <= ~q[2];
    end
    always @(posedge q[2] or posedge rst) begin
        if (rst)
            q[3] <= 1'b0;
       else
            q[3] <= ~q[3];
    end
endmodule
```

OUTPUT



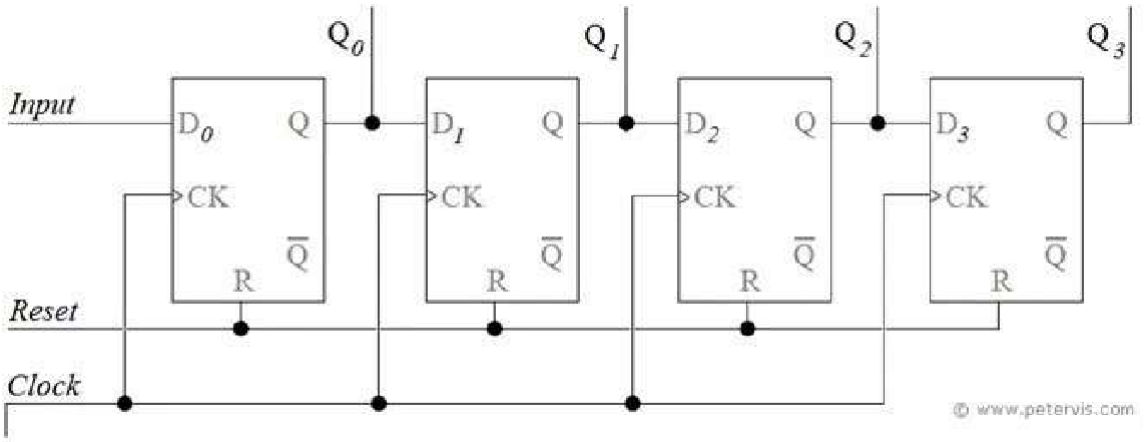
An asynchronous counter, also called a ripple counter, is a sequential circuit in which only the first flip-flop receives the external clock. Each subsequent flip-flop is triggered by the output transition of the previous flip-flop. Therefore, the changes ripple through the counter stages.

Output Waveform Explanation:

The waveform shows the clock, reset, and 4-bit output. After reset is released, the least significant bit toggles with the external clock, while the remaining bits toggle at progressively lower frequencies. The output follows the binary counting sequence from 0000 to 1111 and then repeats.

9 DESIGN, VERILOG HDL CODE & TESTBENCH FOR SHIFT REGISTER :

DESIGN



VERILOG HDL CODE & TESTBENCH

```
`timescale 1ns/1ps

module shift_register_tb;

reg clk;
reg rst;
reg [1:0] mode;
reg [3:0] D;
reg SR, SL;
wire [3:0] Q;

shift_register uut (
    .clk(clk),
    .rst(rst),
    .mode(mode),
    .D(D),
    .SR(SR),
    .SL(SL),
    .Q(Q)
);

// Generate clock with 10 ns period
initial begin
    clk = 0;
    forever #5 clk = ~clk;
end

initial begin
    $dumpfile("shift_register.vcd");
    $dumpvars(0, shift_register_tb);

    $monitor("Time=%0t rst=%b mode=%b D=%b SR=%b SL=%b Q=%b",
             $time, rst, mode, D, SR, SL, Q);

    // Reset
    rst = 1;
    mode = 2'b00;
    D = 4'b0000;
    SR = 0;
    SL = 0;
    #10;

    // Parallel load 1011
    rst = 0;
    mode = 2'b11;
    D = 4'b1011;
    #10;

    // Hold data
    mode = 2'b00;
    #10;

    // Shift right, insert 0
    mode = 2'b01;
    SR = 0;
    #10;

    // Shift left, insert 1
    mode = 2'b10;
    SL = 1;
    #10;

    // Parallel load 1100
    mode = 2'b11;
    D = 4'b1100;
    #10;

    $finish;
end

endmodule
```

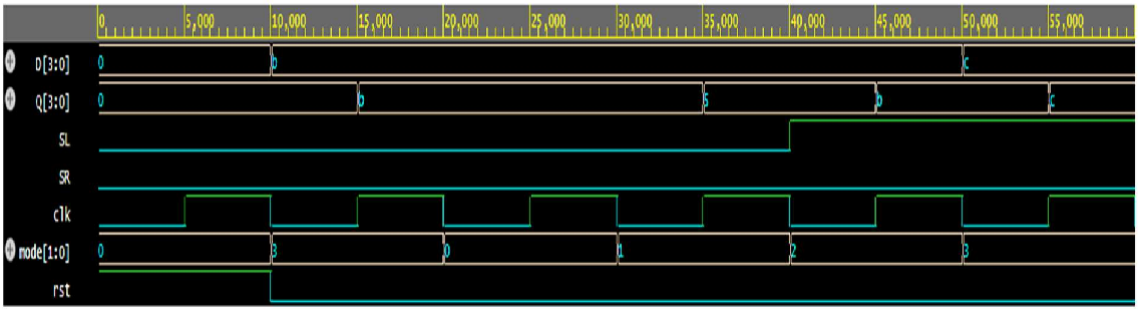
```
module shift_register (
    input clk,
    input rst,
    input [1:0] mode,
    input [3:0] D,
    input SR,
    input SL,
    output reg [3:0] Q
);

always @(posedge clk) begin
    if (rst)
        Q <= 4'b0000;

    else begin
        case (mode)
            2'b00: Q <= Q;              // Hold
            2'b01: Q <= {SR, Q[3:1]}; // Shift right
            2'b10: Q <= {Q[2:0], SL}; // Shift left
            2'b11: Q <= D;              // Parallel load
            default: Q <= Q;
        endcase
    end
end

endmodule
```

OUTPUT



Here is a complete 4-bit shift register design, Verilog HDL code, and testbench that you can run on EDA Playground.

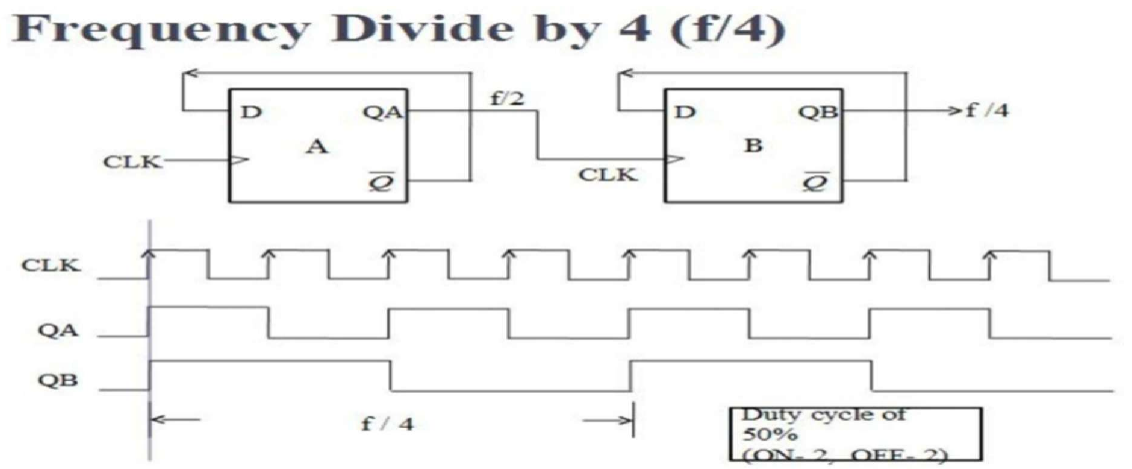
A shift register is a sequential digital circuit that stores binary data and shifts it one bit to the left or right for every active clock edge.

Output Waveform Explanation:

- clk: Controls when the register updates.
- rst: Clears the output to 0000 when high at the active clock edge.
- D[3:0]: Four parallel data inputs.
- SR: Serial bit entering the most significant bit during a right shift.
- SL: Serial bit entering the least significant bit during a left shift.
- Q[3:0]: Four-bit output storing the data.
- always @(posedge clk): Makes the register update only at the rising edge of the clock.
- <=: Non-blocking assignment, commonly used for sequential circuits.

10 DESIGN, VERILOG HDL CODE & TESTBENCH FOR DIVIDE-BY-N COUNTER:

DESIGN



VERILOG HDL CODE & TESTBENCH

```
module tb;
 reg clk, rst;
    wire clk_out;
    divide_by_n uut (
        .clk(clk),
        .rst(rst),
        .clk_out(clk_out)
    );
  initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    initial begin
        $dumpfile("divide_by_n.vcd");
        $dumpvars(0, tb);

        rst = 1;
        #10;

        rst = 0;
        #200;

        $finish;
    end
  initial begin
        $monitor("Time=%0t clk=%b rst=%b clk_out=%b",
                 $time, clk, rst, clk_out);
    end

endmodule
```

```

module divide_by_n (
    input clk,
    input rst,
    output reg clk_out
);

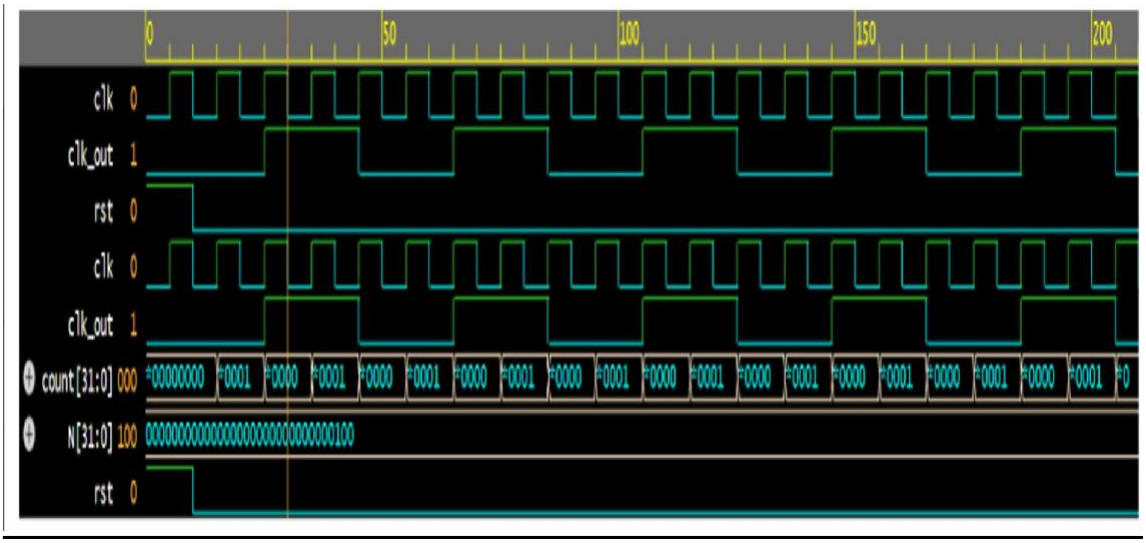
    parameter N = 4;
    integer count;

    always @(posedge clk or posedge rst) begin
        if (rst) begin
            count <= 0;
            clk_out <= 0;
        end
        else begin
            if (count == (N/2)-1) begin
                count <= 0;
                clk_out <= ~clk_out;
            end
            else
                count <= count + 1;
        end
    end

endmodule

```

OUTPUT



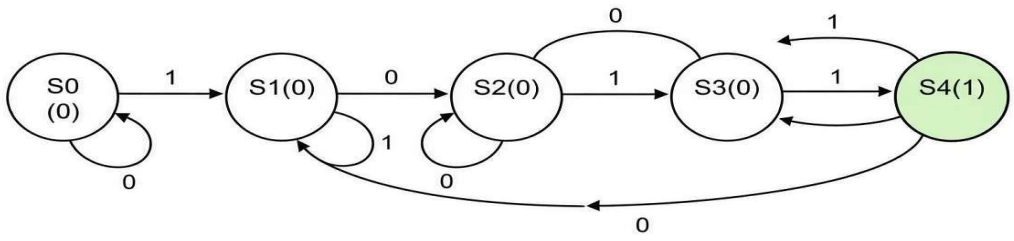
- A Divide-by-N counter divides the input clock frequency by a specified factor. The counter counts clock pulses and toggles the output after a fixed number of pulses.

Output Waveform Explanation:

Initially, the reset signal sets the counter and output to zero. After reset is released, the counter counts the input clock pulses. The output toggles after every two clock pulses, producing a frequency equal to one-fourth of the input clock frequency.

11 DESIGN, VERILOG HDL CODE & TESTBENCH FOR MEALY AND MOORE FSM'S:

MOORE SEQUENCE DETECTER



Grok

VERILOG HDL CODE & TESTBENCH

```
`timescale 1ns/1ps

module tb_moore_detector;

    reg clk, reset, din;
    wire detected;

    moore_detector uut (
        .clk(clk),
        .reset(reset),
        .din(din),
        .detected(detected)
    );

    // Clock generation
    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    // Input sequence: 1011011
    initial begin
        reset = 1;
        din = 0;

        repeat (2) @(negedge clk);
        reset = 0;

        @(negedge clk); din = 1;
        @(negedge clk); din = 0;
        @(negedge clk); din = 1;
        @(negedge clk); din = 1;
```

```
module moore_detector(
    input clk,
    input reset,
    input din,
    output reg detected
);

    parameter S0 = 3'b000,
              S1 = 3'b001,
              S2 = 3'b010,
              S3 = 3'b011,
              S4 = 3'b100;

    reg [2:0] state, next_state;

    // State register
    always @(posedge clk or posedge reset) begin
        if (reset)
            state <= S0;
        else
            state <= next_state;
    end

    // Next-state logic
    always @(*) begin
        case (state)
            S0: next_state = din ? S1 : S0;
            S1: next_state = din ? S1 : S2;
            S2: next_state = din ? S3 : S0;
            S3: next_state = din ? S4 : S2;
            S4: next_state = din ? S1 : S2;
```

```
        @(negedge clk); din = 1;
        @(negedge clk); din = 0;
        @(negedge clk); din = 1;
        @(negedge clk); din = 1;
        @(negedge clk); din = 0;
        @(negedge clk); din = 1;
        @(negedge clk); din = 1;

        @(negedge clk);
        $finish;
    end

    // Generate waveform file
    initial begin
        $dumpfile("moore_wave.vcd");
        $dumpvars(0, tb_moore_detector);
    end

endmodule
```

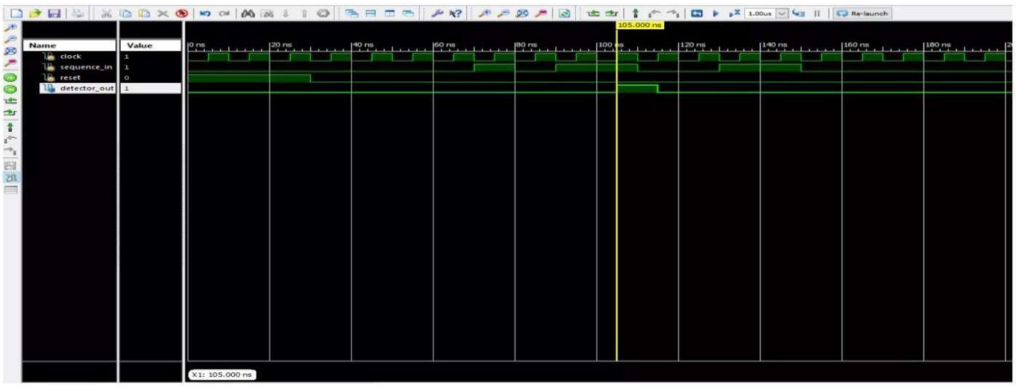
```
    // Next-state logic
    always @(*) begin
        case (state)
            S0: next_state = din ? S1 : S0;
            S1: next_state = din ? S1 : S2;
            S2: next_state = din ? S3 : S0;
            S3: next_state = din ? S4 : S2;
            S4: next_state = din ? S1 : S2;

            default: next_state = S0;
        endcase
    end

    // Moore output logic
    always @(*) begin
        detected = (state == S4);
    end

endmodule
```

OUTPUT



- A Moore machine is a type of finite state machine (FSM) used in digital circuits, where the output depends only on the current state. It works by transitioning between states according to the input received at each clock cycle.

4. Expected output waveform

The input sequence is `1011011`. It contains two overlapping occurrences of `1011`.

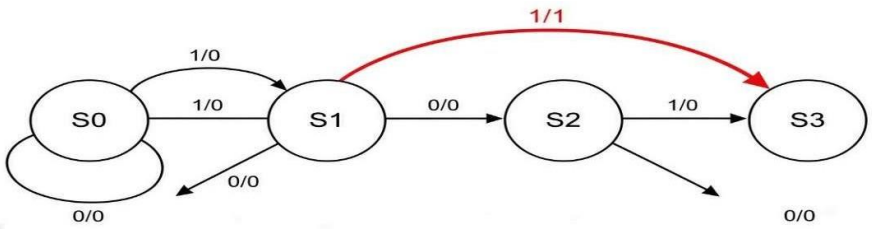
Illustrative timing diagram — each column represents one clock cycle after reset.

Signal	1	2	3	4	5	6	7
clk	1	1	1	1	1	1	1
din	1	0	1	1	0	1	1
detected	0	0	0	1	0	0	1

Output Waveform Explanation:

The waveform shows the clock, reset, input, and output signals during simulation. The clock controls state transitions, and reset initializes the machine. When the required input sequence is detected, the machine enters a specific state and the output becomes 1. Otherwise, the output remains 0, demonstrating the correct operation of the Moore machine.

MEALY SEQUENCE DETECTOR



VERILOG HDL CODE & TESTBENCH

```
`timescale 1ns/1ps

module tb_mealy_detector;

    reg clk, reset, din;
    wire detected;

    mealy_detector uut (
        .clk(clk),
        .reset(reset),
        .din(din),
        .detected(detected)
    );

    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    initial begin
        reset = 1;
        din = 0;

        repeat (2) @(negedge clk);
        reset = 0;

        @(negedge clk); din = 1;
        @(negedge clk); din = 0;
        @(negedge clk); din = 1;
        @(negedge clk); din = 1;
        @(negedge clk); din = 0;
        @(negedge clk); din = 1;
```

```
module mealy_detector(
    input clk,
    input reset,
    input din,
    output reg detected
);

    parameter S0 = 2'b00,
              S1 = 2'b01,
              S2 = 2'b10,
              S3 = 2'b11;

    reg [1:0] state, next_state;

    // State register
    always @(posedge clk or posedge reset) begin
        if (reset)
            state <= S0;
        else
            state <= next_state;
    end

    // Next-state and output logic
    always @(*) begin
        next_state = S0;
        detected = 0;

        case (state)
            S0: begin
                if (din)
                    next_state = S1;
                else
```

```
        clk = 0;
        forever #5 clk = ~clk;
    end

    initial begin
        reset = 1;
        din = 0;

        repeat (2) @(negedge clk);
        reset = 0;

        @(negedge clk); din = 1;
        @(negedge clk); din = 0;
        @(negedge clk); din = 1;
        @(negedge clk); din = 1;
        @(negedge clk); din = 0;
        @(negedge clk); din = 1;
        @(negedge clk); din = 1;

        @(negedge clk);
        $finish;
    end

    initial begin
        $dumpfile("mealy_wave.vcd");
        $dumpvars(0, tb_mealy_detector);
    end

endmodule
```

```
                next_state = S0;
            end

            S1: begin
                if (din)
                    next_state = S1;
                else
                    next_state = S2;end

            S2: begin
                if (din)
                    next_state = S3;
                else
                    next_state = S0;
            end

            S3: begin
                if (din) begin
                    detected = 1;
                    next_state = S1;
                end
                else
                    next_state = S2;
            end

            default: begin
                next_state = S0;
                detected = 0;
            end
        endcase
    end
```

OUTPUT



- A **Mealy machine** is a finite state machine (FSM) in which the output depends on both the present state and the current input. Unlike a Moore machine, a Mealy machine can produce an output immediately when the required input bit arrives, without needing an additional state to indicate detection.

Add these signals to your simulator waveform window in the same order as your screenshot:

Signal	Description
`z`	Output — detects sequence `1011`
`x`	Serial input
`clk`	Clock signal with 10 ns period
`reset`	Initializes the FSM

For the input `1011011`, the output `z` pulses high when the fourth and seventh input bits complete the pattern `1011`.

Output Waveform Explanation:

The waveform represents the behavior of the Mealy machine during simulation. The clock controls state transitions, reset initializes the machine, and the input determines the next state and output. The output changes according to the current state and input, demonstrating the correct operation of the circuit.

12 DESIGN, VERILOG HDL CODE & TESTBENCH FOR RANDOM SEQUENCE GENERATOR:

VERILOG HDL CODE & TESTBENCH

```
module tb;
  reg clk, rst;
    wire [3:0] Q;
  random_sequence uut (
        .clk(clk),
        .rst(rst),
        .Q(Q)
    );
  initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end
  initial begin
        $dumpfile("random_sequence.vcd");
        $dumpvars(0, tb);

        rst = 1;
        #10;

        rst = 0;
        #160;

        $finish;
    end

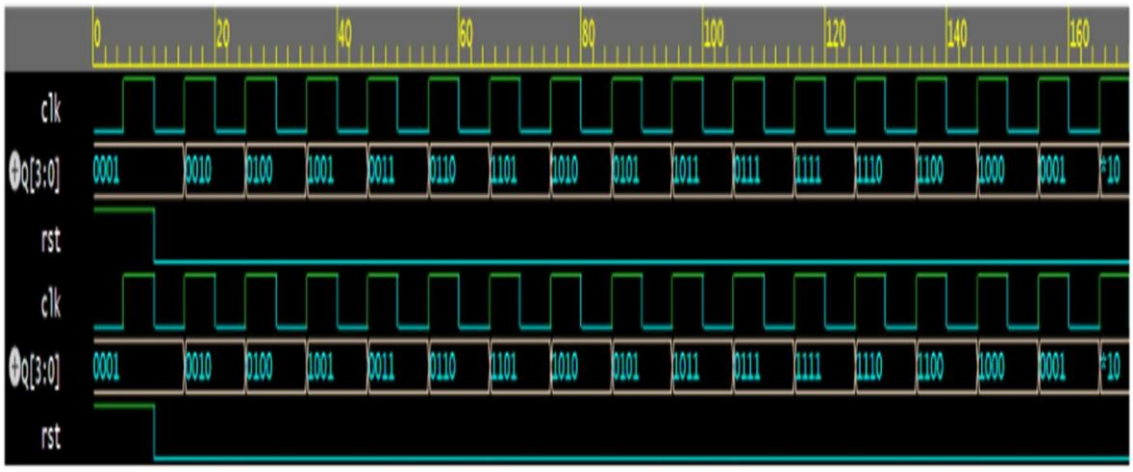
    initial begin
        $monitor("Time=%0t clk=%b rst=%b Q=%b",
                 $time, clk, rst, Q);
    end
endmodule
```

```
module random_sequence (
    input clk,
    input rst,
    output reg [3:0] Q
);

    always @(posedge clk or posedge rst) begin
        if (rst)
            Q <= 4'b0001;
        else
            Q <= {Q[2:0], Q[3] ^ Q[2]};
    end

endmodule
```

OUTPUT



- random sequence generator generates a sequence of binary numbers. In digital circuits, a Linear Feedback Shift Register (LFSR) is commonly used to generate a pseudo-random sequence.
- An LFSR consists of flip-flops connected in a shift-register arrangement and an XOR gate for feedback. At every positive edge of the clock, the register shifts its bits and calculates a new input bit using XOR logic.

Output waveform Explanation:

Initially, the reset signal is activated, and the output register is set to 0001. After the reset is released, the circuit generates a new output at every rising edge of the clock. The XOR feedback logic calculates the next bit, while the remaining bits shift to their new positions.

he generated 4-bit sequence is:

0001 → 0011 → 0111 → 1111 → 1110 → 1101 → 1010 → 0101 → 1011 → 0110 → 1100 → 1001 → 0010 → 0100 → 1000 → 0001